УКРАЇНСЬКИЙ ДЕРЖАВНИЙ УНІВЕРСИТЕТ ІМЕНІ МИХАЙЛА ДРАГОМАНОВА

ФАКУЛЬТЕТ МАТЕМАТИКИ, ІНФОРМАТИКИ ТА ФІЗИКИ

Кафедра комп'ютерної та програмної інженерії

ЗВІТ ПРО ВИКОНАНУ РОБОТУ ДО ЗАВДАННЯ «ДОСЛІДЖЕННЯ БІБЛІОТЕК»

з дисципліни ««Розробка мобільних застосунків»»

Виконав/ла: студент/ка 4 курсу, групи 41ІПЗ,

спеціальності 121 Інженерія програмного забезпечення

(шифр і назва спеціальності)

Славінська Софія

Викладач: кандидат педагогічних наук, доцент

доцент кафедри комп'ютерної та програмної інженерії

Шевчук Б.В.

Київ – 2026

C1 - Internal use

Зміст

Введення……..3

Актуальність тематики……3

Мета ……...3

Завдання……3

Методи……...3

Основна частина ……...4

Етапи реалізації ……...4

Висновки……5

Проблеми і труднощі 5

Досягнутий результат ……5

Список використаної літератури……...6

C1 - Internal use

Введення

Актуальність тематики:

Розробка мобільних застосунків

Мета: *(мета лабораторної роботи)*

Дослідити архітектуру, функціональність та особливості впровадження сучасної мобільної бібліотеки (SDK) у проєкті, провести порівняльний аналіз альтернативних рішень та реалізувати демонстраційний прототип із використанням обраної бібліотеки.

Завдання: *(виконання перелічених завдань із пункту «Хід роботи» практичної роботи)*

1. Підготовка до виконання роботи
2. Виконання завдання, які зазначені у пунктах «Хід роботи»
3. Оформлення звіту про виконану роботу та проведення його захисту.

Методи: *(які були використані для їх вирішення)*

Пошук та аналіз інформації

C1 - Internal use

Основна частина

Етапи реалізації: *(вкажіть всі використані ресурси: дослідження, досліди, залучення інших джерел інформації, виконані пункти завдань)*

1. Отримання варіанту
    a. Я обрала варіант - Завантаження зображень
    b. Я створила репозиторій для проєкту https://github.com/SofiiaSlavinska/Coil-SDK-Analysis

```
D:\Desktop\Розробка мобільних застосувань>git clone https://github.com/SofiiaSlavinska/Coil-SDK-Analysis
Cloning into 'Coil-SDK-Analysis'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

2. Теоретичне дослідження

На цьому етапі мною було проведено всебічне дослідження обраної технології — бібліотеки Coil (Coroutine Image Loader), а також виконано порівняльний аналіз існуючих альтернатив для визначення доцільності використання даного SDK у сучасному мобільному розробленні.

Для глибокого розуміння інструментарію я опрацювала офіційну документацію проєкту (разділи *Getting Started* та *Full Guide*). Основну увагу було зосереджено на:

- Вивченні API: Аналіз ключових класів, таких як ImageLoader та ImageRequest.
- Практичних прикладах: Мною було проаналізовано відкриті репозиторії на GitHub для вивчення реальних сценаріїв впровадження Coil у проєкти з архітектурою Clean Architecture та MVVM.
- Мультимедійних ресурсах: Перегляд профільних технічних конференцій та відео-туторіалів дозволив зрозуміти тонкощі конфігурації бібліотеки для роботи з різними типами джерел даних (URL, ресурси, файлова система).

C1 - Internal use

Для об’єктивної оцінки Coil я склала перелік із 5 основних конкурентів: Glide, Picasso, Fresco, Ktor Client та Landscapist. Аналіз проводився за такими критеріями: ліцензія, вплив на розмір APK-файлу, активність спільноти та якість документації.

Результати порівняння:

- Picasso/Glide: Хоча ці бібліотеки є галузевими стандартами, вони побудовані на застарілих механізмах зворотних викликів (callbacks).
- Coil: Виявився найбільш сучасним рішенням, оскільки він є «Kotlin-first» бібліотекою, що використовує Coroutines для управління асинхронними потоками.

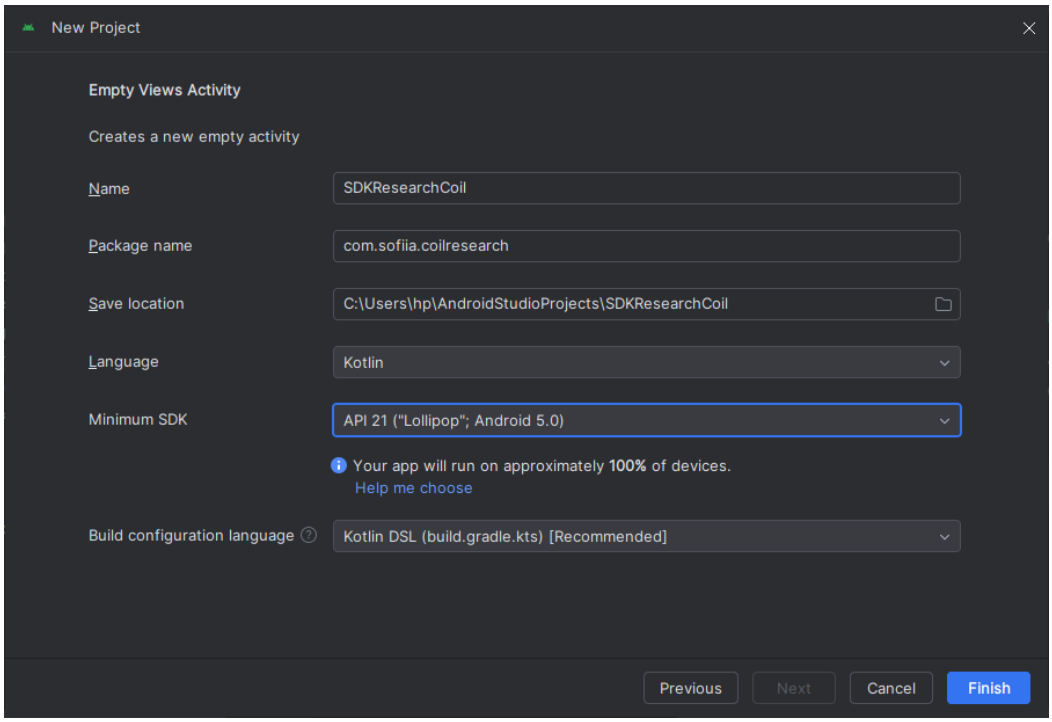
Мною було сформовано детальну порівняльну таблицю (див. додаток COMPARISON.md), яка демонструє переваги Coil у швидкості ініціалізації та нативній підтримці форматів SVG та GIF.

Окрему увагу в межах етапу було приділено внутрішній архітектурі бібліотеки:

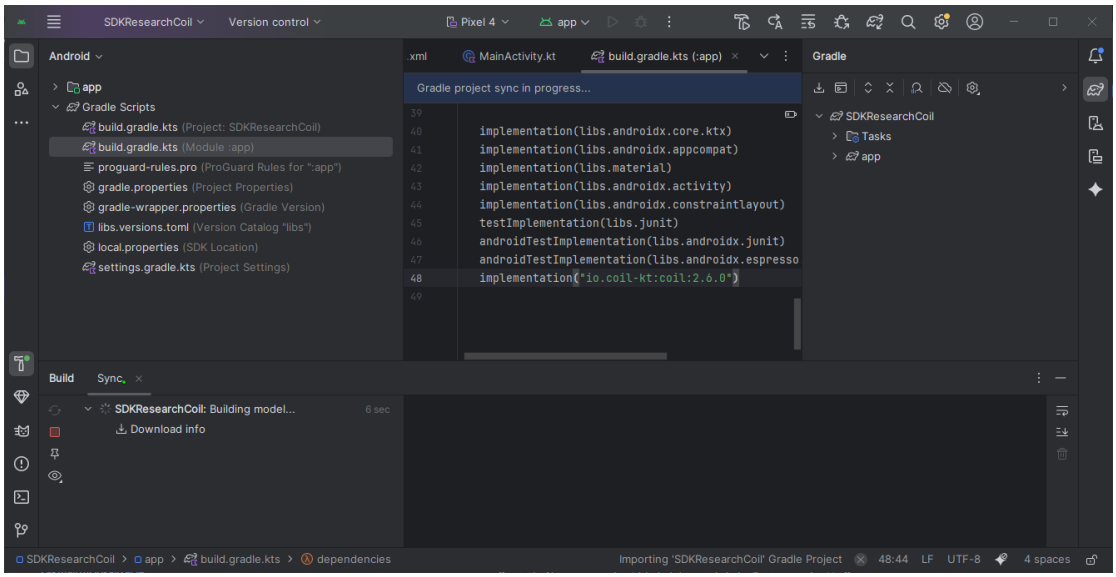
1. Патерни проєктування: Досліджено використання патерну Builder для створення запитів та Singleton для управління глобальним завантажувачем зображень.
2. Життєвий цикл: Проаналізовано механізм автоматичного скасування запитів при знищенні життєвого циклу компонента (Activity/Fragment), що запобігає витокам пам'яті.
1. Робота з пам'яттю та потоками: Вивчено стратегію трирівневого кешування (Memory cache, Disk cache, Network) та інтеграцію з OkHttp для оптимізації мережевих запитів.

3. Практична реалізація
    a. Спочатку я створила новий проєкт

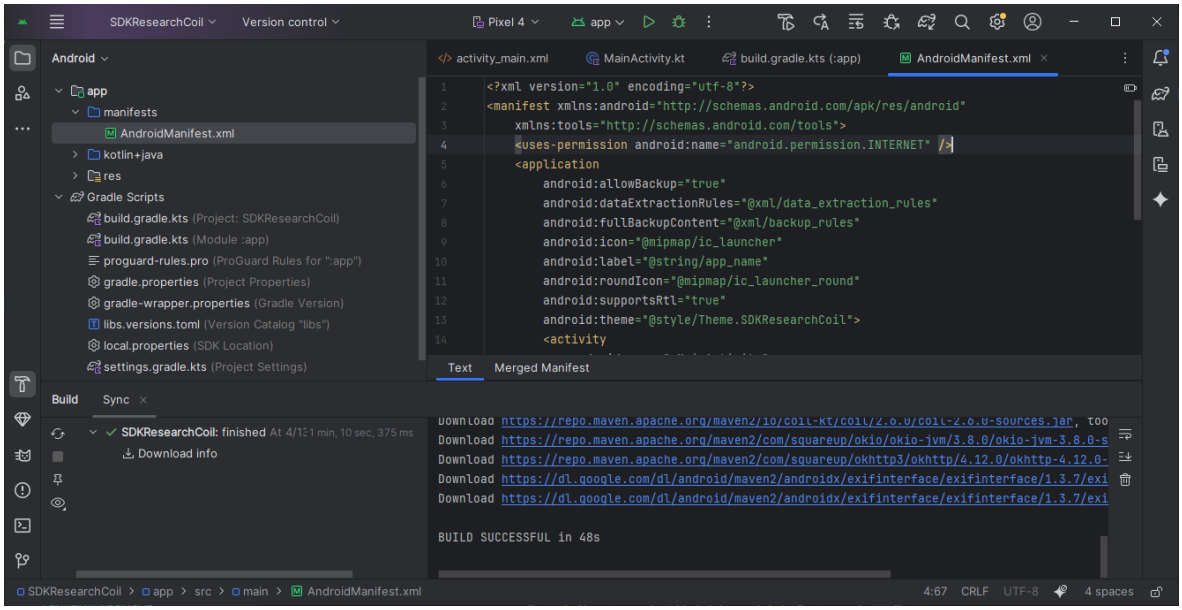
C1 - Internal use



b. Я додала нову dependency Coil

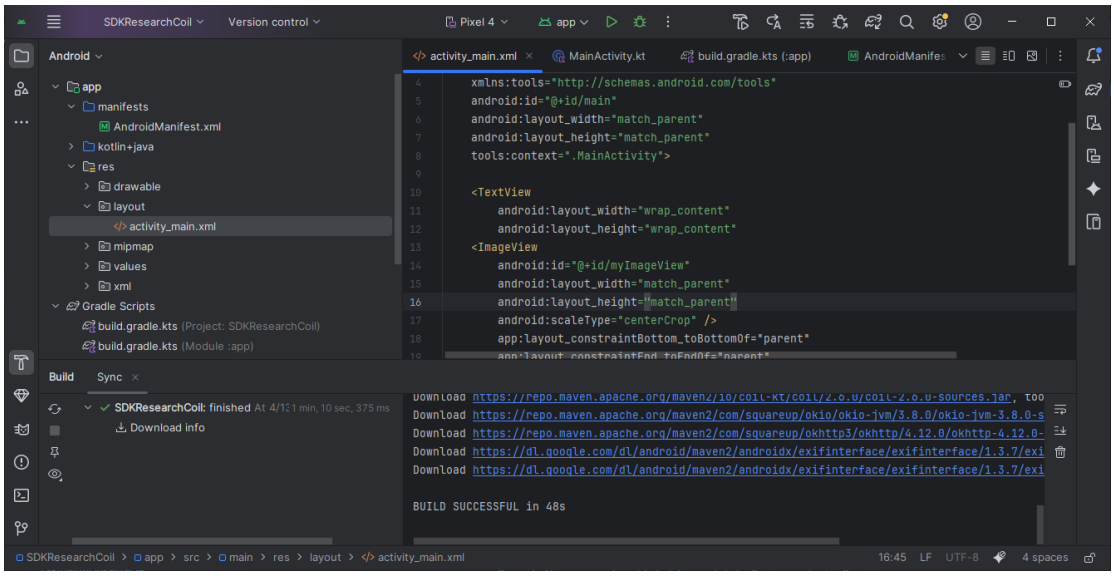


c. Я додала дозвіл на інтернет

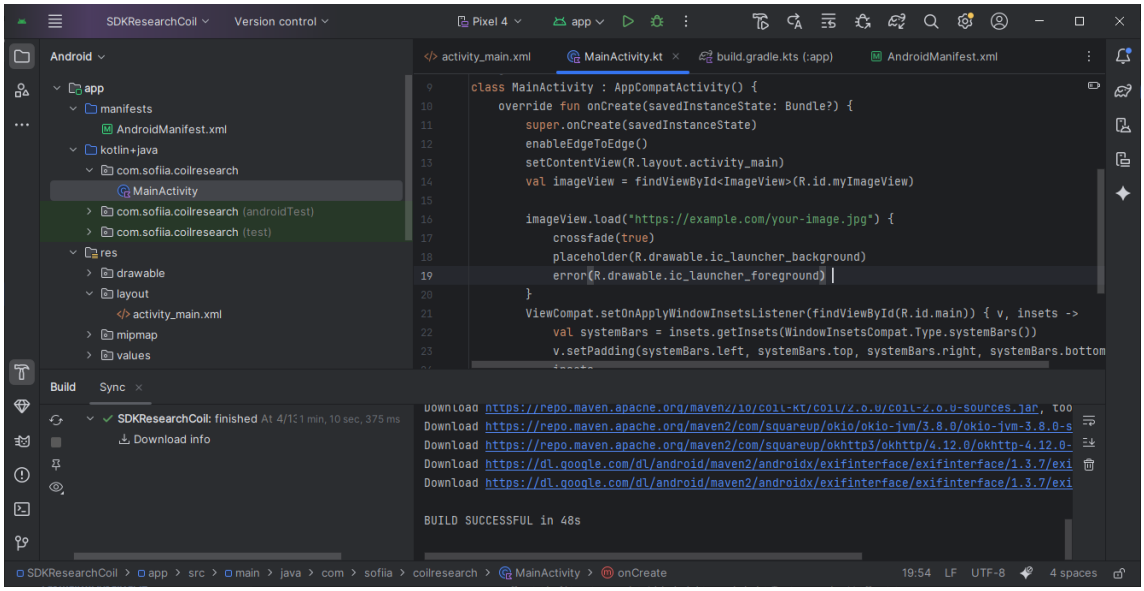


d. Додаю ImageView

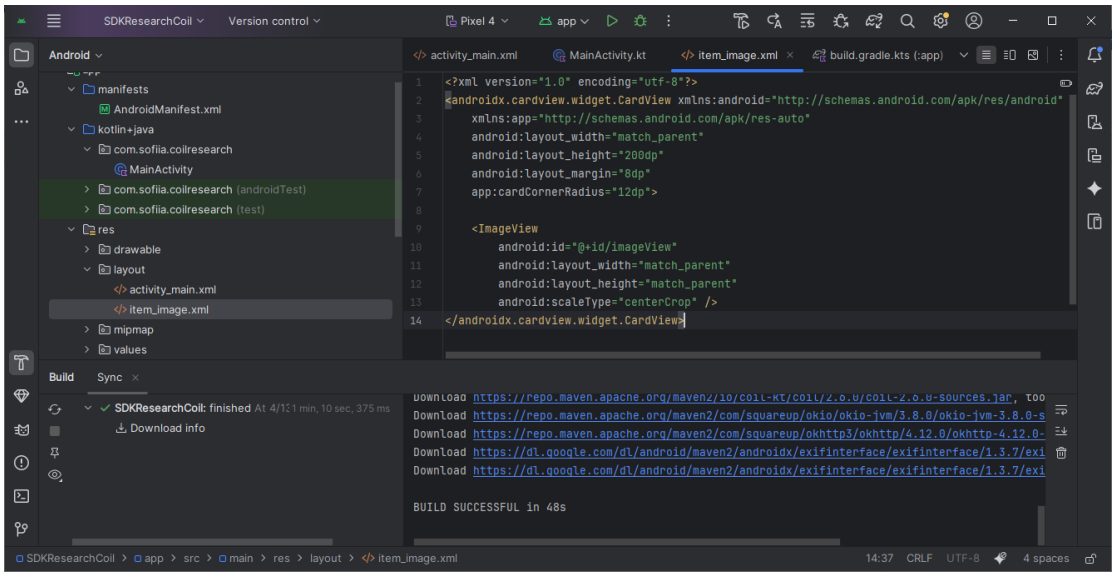
C1 - Internal use



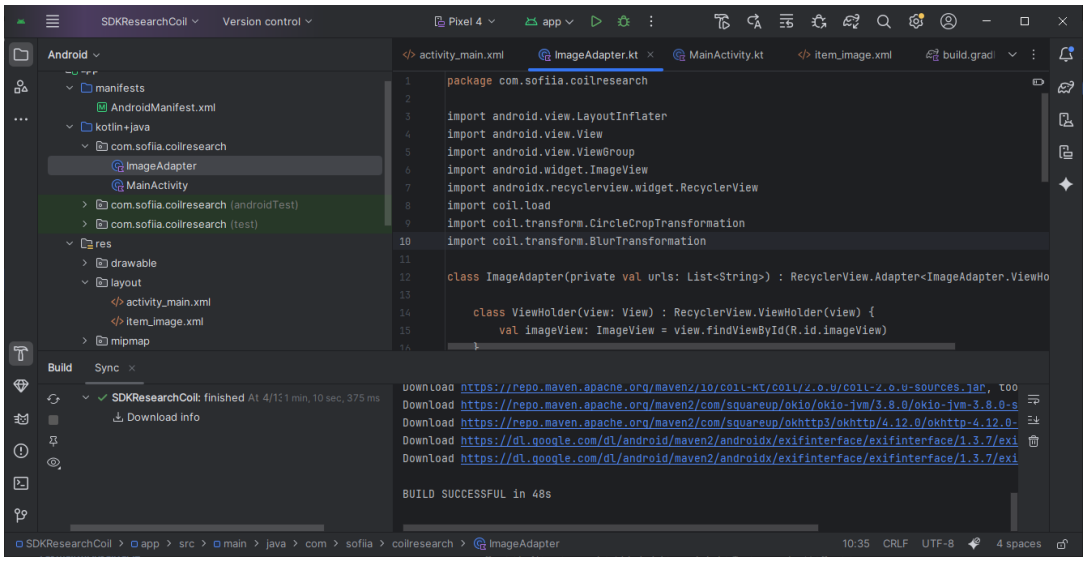
e. Я додала код завантаження



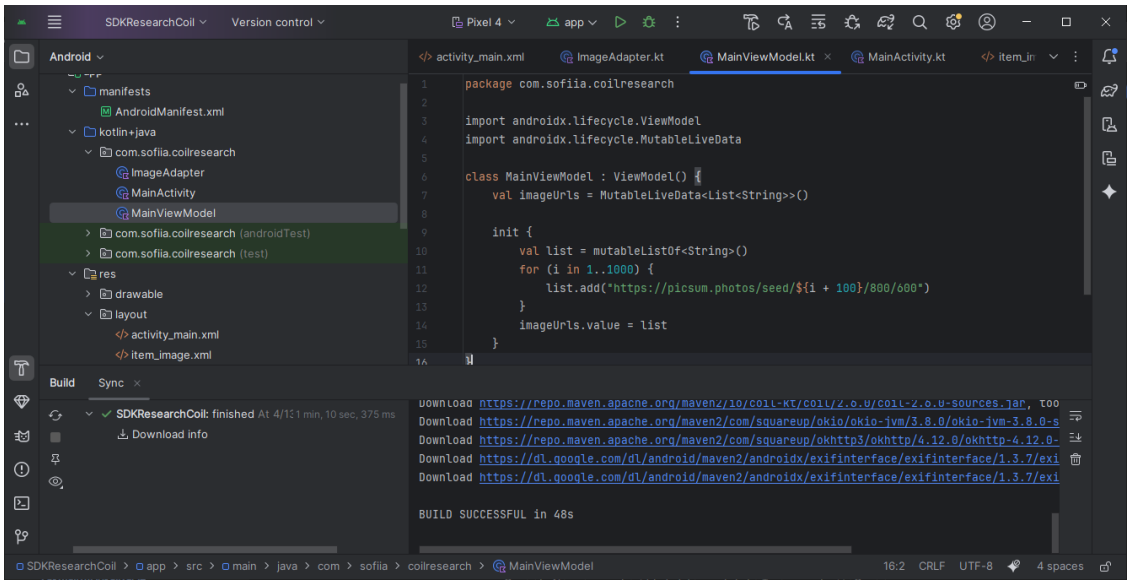
f. Я додала файл розмітки



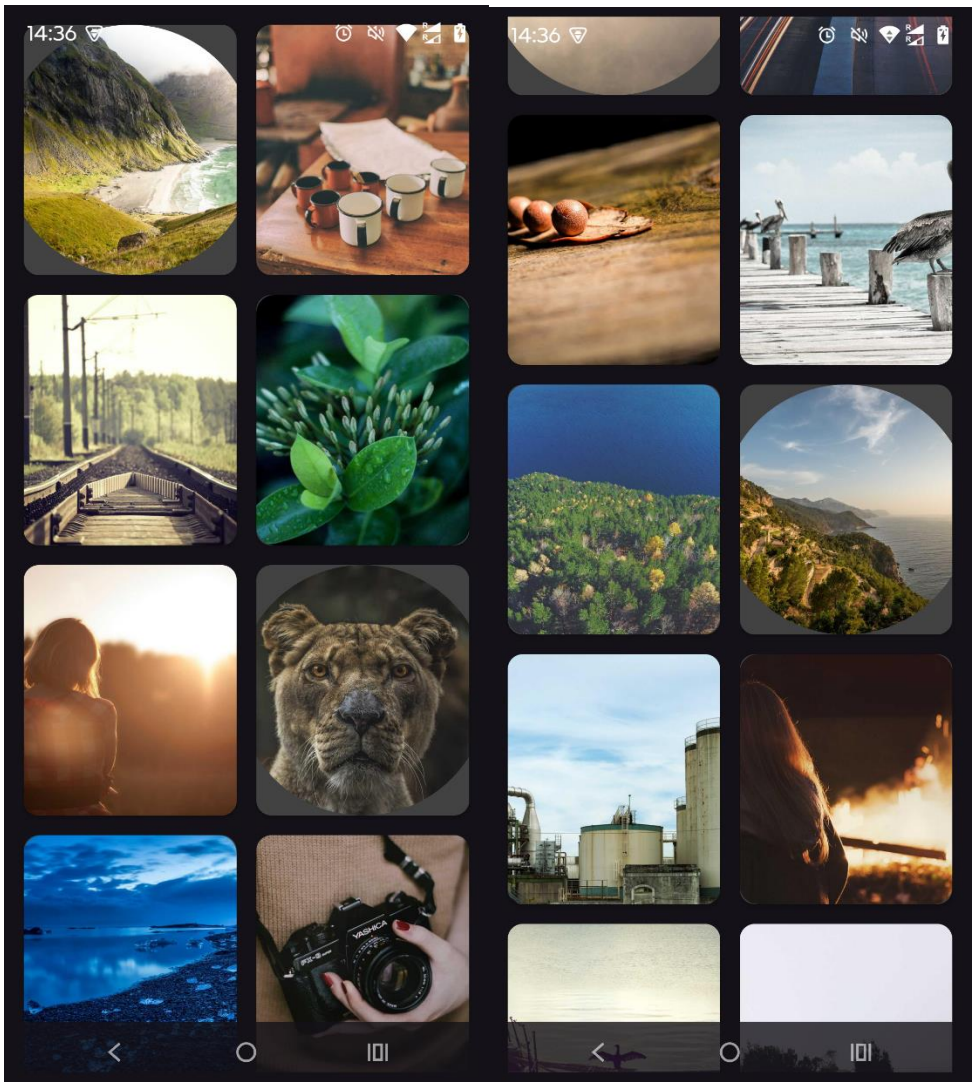
g. Я додала новий клас ImageAdapter



h. Я додала новий клас MainViewModel

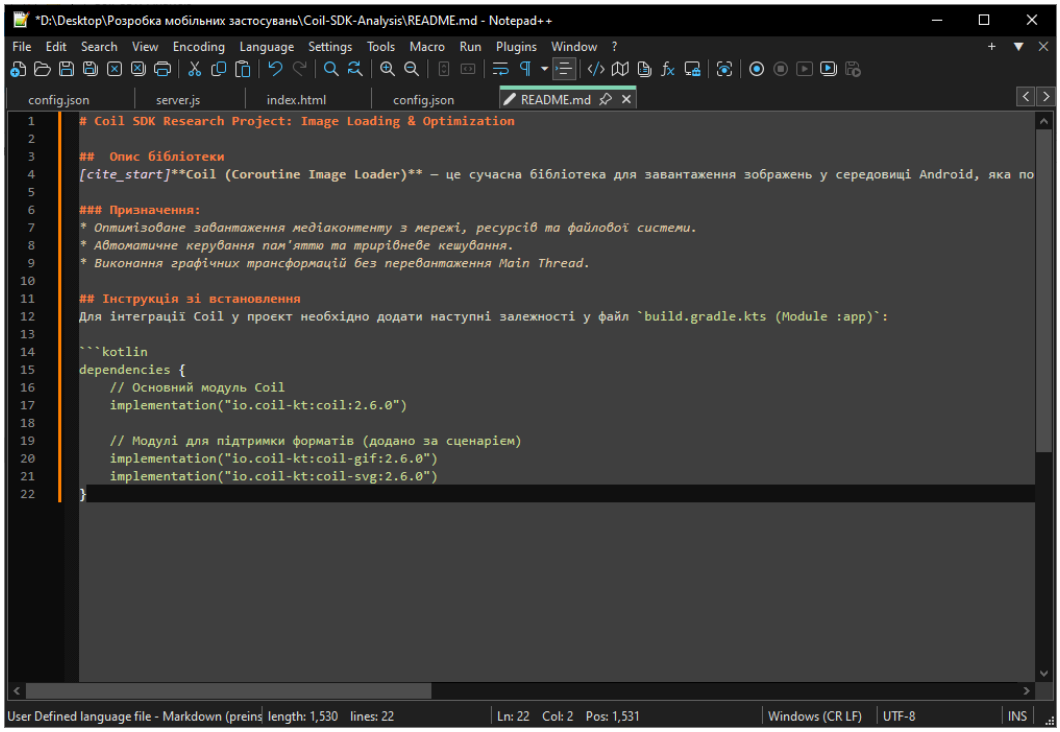


4. Тестування та профілювання
   a. Спочатку виникали помилки, однією з яких був розлад версій. Я це виправила підвищивши SDK з 21 до 23
   b. При запуску застосунок відображає перелік фото з інтернету у різному форматі. Список може скролитись. Проблем не було виявлено

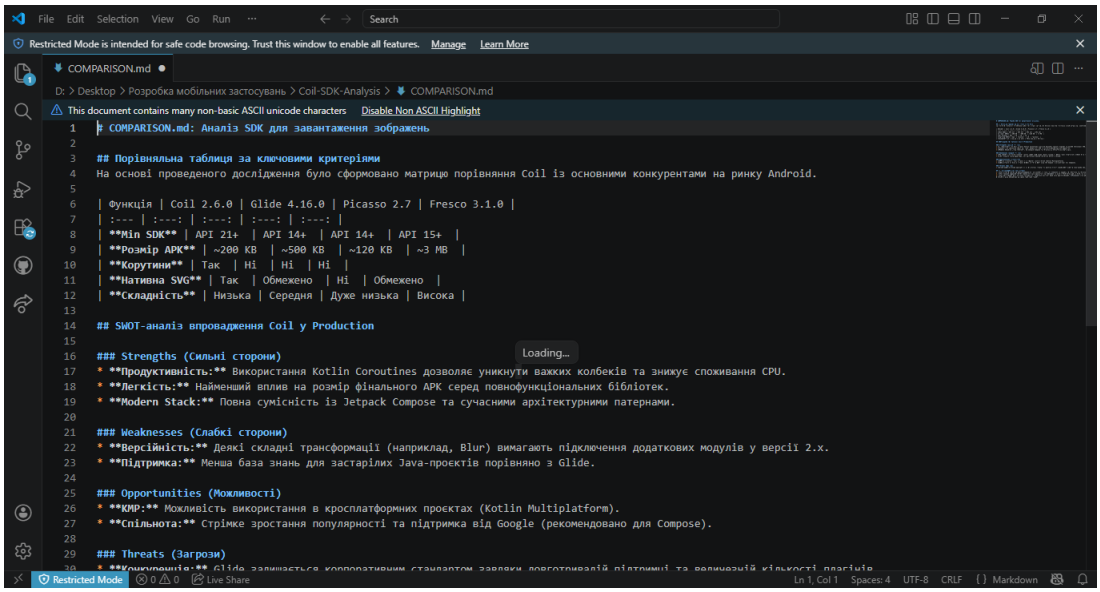


5. Документація

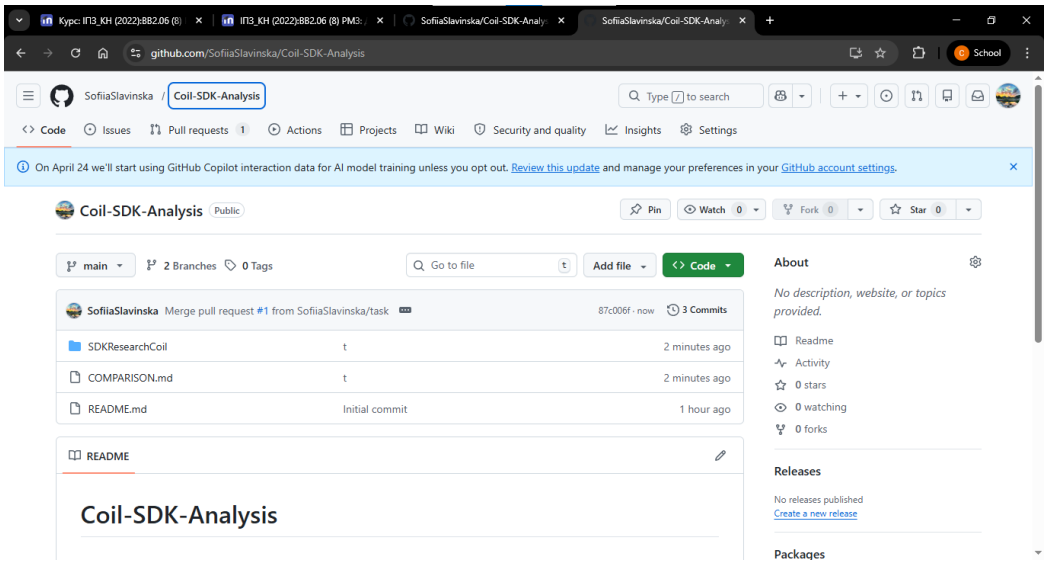
   a. Я створила Readme file



   b. Я створила COMPARISON.md файл



c. Я запаблішила код та файли на репозиторій:



6. Code Review

a. Я взяла на код ревю репозиторій Богдана Шевцова - https://github.com/sheva64/RoomLibrary

Провівши детальний аналіз репозиторію, я можу виділити сильні сторони реалізації, критично оцінити поточні недоліки та визначити вектори для подальшого вдосконалення архітектури.

У проєкті чітко дотримано принципу розділення відповідальності

- Рівень даних повністю ізольований у NoteRepository, що є єдиною точкою входу для даних.
- NoteViewModel успішно забезпечує стійкість даних до змін конфігурації (наприклад, при повороті екрана).

- Клас NoteDatabase реалізовано за патерном Singleton з використанням volatile та synchronized, що гарантує thread-safety при зверненні до бази.
- SQL-запити в NoteDao валідуються на етапі компіляції, а використання LiveData забезпечує реактивне оновлення інтерфейсу без необхідності вручну оновлювати список після кожної операції.
- Всі важкі операції запису та видалення винесені у фоновий потік через ExecutorService (пул з 4 потоків), що дозволяє уникнути блокування UI та помилок ANR.
- Проєкт має детальний README.md з інструкціями та скріншотами.
- Файл COMPARISON.md містить глибокий аналіз альтернатив (Realm, ObjectBox, SQLDelight), що підкреслює обґрунтованість вибору Room для даного стека.
- Реалізовано 5 Unit-тестів на базі In-Memory бази даних, що покривають як базовий CRUD, так і просунуті функції пошуку та сортування.

1. Використання застарілого підходу до оновлення списків:
    - В NoteAdapter я використовую notifyDataSetChanged() для оновлення списку. Це неефективно для великих списків, оскільки перемальовується весь RecyclerView.
    - *Рекомендація:* Впровадити DiffUtil, який обчислює різницю між списками та оновлює лише змінені елементи.
2. Локальна фільтрація vs Запити до БД:
    - Зараз пошук реалізовано шляхом локальної фільтрації копії списку в адаптері. Хоча це працює швидко на малих обсягах, це нівелює перевагу SQL-запиту searchNotes, який я вже прописав у DAO.
    - *Рекомендація:* Використовувати searchNotes через ViewModel, щоб фільтрація відбувалася на рівні бази даних.
3. Використання Java замість Kotlin:

C1 - Internal use

    - Хоча Java є надійною, сучасним стандартом Android-розробки є Kotlin.
    - *Рекомендація:* Міграція на Kotlin дозволила б використовувати Coroutines замість ExecutorService та Flow замість LiveData, що зробило б код більш лаконічним та безпечним (Null Safety).
7. Підготовка до здачі
    a. Посилання на відео-запис: https://drive.google.com/drive/folders/1Wf0c3euZCN9avDjqrFw1ZzJKePEANp1U?dmr=1&ec=wgc-drive-globalnav-goto

C1 - Internal use

Висновки

Досягнутий результат: *(сформуйте висновок по закінчення виконання завдань лабораторної роботи)*

У межах роботи я дослідила архітектуру, функціональність та особливості впровадження сучасної мобільної бібліотеки (SDK) у проєкті, провести порівняльний аналіз альтернативних рішень та реалізувати демонстраційний прототип із використанням обраної бібліотеки.

C1 - Internal use

Список використаної літератури:.

1. Шейко, В. М. Організація наукових досліджень : методика, досвід, поради / В. М. Шейко, Н. М. Кушнаренко. – 4-те вид., допов. – К. : Знання, 2011. – 254 с.
2. IEEE. Software Requirements Specification Template [Electronic resource]. – Mode of access: https://ieee.org – Date of access: 02.04.2026.
3. • Microsoft Learn. Requirements and planning in software projects [Electronic resource]. – Mode of access: https://learn.microsoft.com/en-us/devops/ – Date of access: 02.04.2026.
4. Responsive layout grid: https://m2.material.io/design/layout/responsive-layout-grid.html#breakpoints – Date of access: 02.04.2025
5. Guide to app architecture: https://developer.android.com/topic/architecture – Date of access: 02.04.2025

C1 - Internal use